



Zagadka

24Pomorzanka04, grupa A, dzień 3. Pamięć: 128 MB. Limit czasu: 15 s.**05.04.2023**

Małgosia zadała Ci następującą zagadkę: wybrała pewien ciąg zerojedynekowy długości n i musisz go zgadnąć. Pozwoliła Ci zadać pewną liczbę pytań następującej postaci: możesz wybrać pewien ciąg zerojedynekowy, również długości n , i przekazać go koleżance. Małgosia odpowie Ci, na ilu pozycjach oba ciągi są zgodne, ale tylko, jeżeli są zgodne na wszystkich lub dokładnie na połowie miejsc.

Małgosia jest uczciwa: wybiera ciąg na początku, a nie dostosowuje go do pytań, które jej zadajesz.

Zadanie

Twoim zadaniem jest napisać funkcję `void zagadka()`. Ta funkcja odpowiada za próbę zgadnięcia jednego ciągu. Twój program nie powinien zawierać funkcji `main()`. Nie korzystaj także ze standardowego wejścia i wyjścia. Zamiast tego, komunikacja będzie się odbywała za pomocą funkcji dostarczonych przez bibliotekę. Aby użyć biblioteki, należy wpisać na początku programu `#include "czag.h"`.

Biblioteka udostępnia następujące funkcje i procedury:

- `int inicjuj()` — zwraca parzystą liczbę n . Powinna zostać użyta w funkcji `zagadka` dokładnie raz, na początku.
- `int zapytaj(vector<bool> t)` — zadaje bibliotece pytanie. Odpowiedzią jest 0 (oznaczające brak odpowiedzi), $n/2$ lub n , zgodnie z opisem.
- `void odpowiedz(vector<bool> t)` — informuje bibliotekę, że odpowiedzią w zagadce jest ciąg przekazywany przez tę funkcję jako argument.

Ocenianie

q oznacza liczbę wywołań funkcji `zagadka()` w pojedynczym teście. Każdy test ma również parametry p_1 i p_2 . Jeżeli wykonasz z zapytań w pojedynczym wywołaniu funkcji `zagadka()`, to wywołanie zostanie ocenione na $\frac{p_2 - z}{p_2 - p_1}$, przy czym wartość zostanie obcięta do przedziału $[0, 1]$. Wynikiem za test (otrzymanym ułamkiem liczby punktów) jest minimum z wyników dla poszczególnych zagadek.

Na przykład, jeżeli test ma parametry $p_1 = 10$ i $p_2 = 20$, w pierwszym wywołaniu użyjesz 9 zapytań, a w drugim 16, to dostaniesz 40% punktów za test.

Zestaw testów dzieli się na następujące podzadania.

Podzadanie	Warunki	Punktacja
1	$q = 100, n = 200, p_1 = p_2 = 10000$	7
2	$q = 500, n = 1000, p_1 = 2000, p_2 = 5000$	12
3	$q = 500, n = 1000, p_1 = 2000, p_2 = 2010$	15
4	$q = 500, n = 1000, p_1 = 1300, p_2 = 2000$	42
5	$q = 500, n = 1000, p_1 = p_2 = 1300$	24

Przykładowe wykonanie

Poniższa tabela zawiera przykładowy ciąg pytań do biblioteki prowadzący do odgadnięcia ciągu. Ten ciąg jest identyczny z ciągiem z testu `zag0` (w tym teście jest tylko jedno zapytanie).

Nr pytania	Wywołanie	Odpowiedź	Wyjaśnienie
	<code>inicjuj</code>	2	$n = 2$, odgadywany ciąg to 1,1
1	<code>zapytaj({1,0})</code>	1	ciągi zgadzają się na jednej pozycji
2	<code>zapytaj({0,0})</code>	0	ciągi nie zgadzają się na żadnej pozycji
	<code>odpowiedz({1,1})</code>		odgadujemy, że ciąg to 1,1

Eksperymenty

W plikach powinna znajdować się przykładowa biblioteka pozwalające na sprawdzenie poprawności formalnej rozwiązania. Biblioteka przykładowa wczytuje ze standardowego wejścia najpierw liczbę zapytań q , następnie opisy ciągów w formacie:

- w pierwszym wierszu liczba n — długość ciągu,
- w drugim wierszu n liczb poodzielanych pojedynczymi odstępami — elementy ciągu.

Przykładowe wejście do biblioteki znajdziesz w pliku `zag0.in`. Po zakończeniu działania biblioteka wypisuje, czy odpowiedź była poprawna, oraz liczbę zadanych pytań.

W archiwum z biblioteką powinny znajdować się również przykładowe rozwiązanie `zag.cpp` korzystające z biblioteki. Rozwiązania te nie znajdują poprawnej odpowiedzi.

Do kompilacji rozwiązania wraz z biblioteką służy polecenie `g++ czag.cpp zag.cpp -o zag`.

Plik z rozwiązaniem i biblioteka powinny znajdować się w tym samym katalogu.